ProfsAgent Phase 2 · Deliverable C — Learning Resources and Expert Speakers

Owner: Vihan · **Status:** PLAN v0.1 for review (24 Oct 2026) · **Builds on:** *ProfsAgent Phase 2 — Architectural Design* (draft v0.1, 21 Oct 2026) and the Phase 1 pipeline.

---

0. Summary

Phase 2 helps a professor **deliver** a course that Phase 1 designed. The motivating problem: new courses must be launched fast, and a professor may be able to teach only part of the 13 weeks well. Deliverable C gives the professor, **per lecture**:

1. a **lecture brief** (what the lecture covers, what it builds on, what it feeds into) and a simple question: *"Will you teach this yourself, or do you want help?"*;
2. for lectures where help is wanted, a **short, ranked, explained list of learning resources** (lecture videos, readings, interactive material, practice) that cover exactly that lecture's topics;
3. where the professor wants someone else to teach it, a **ranked, explained list of experts and practitioners worldwide** who could deliver that lecture, with a ready guest-lecture brief.

Everything is **retrieved, not invented** (every link is checked to exist), **ranked by transparent criteria** (coverage of the lecture's topics first), **capped** so the professor is not overloaded, **checked** before the professor sees it, and **approved by the professor** before students see it. It plugs into the Phase 2 architecture as aid services behind the common interface, reusing the Phase 1 machinery (LLM client, validation engine, review gates, Neo4j).

---

1. Scope and boundaries

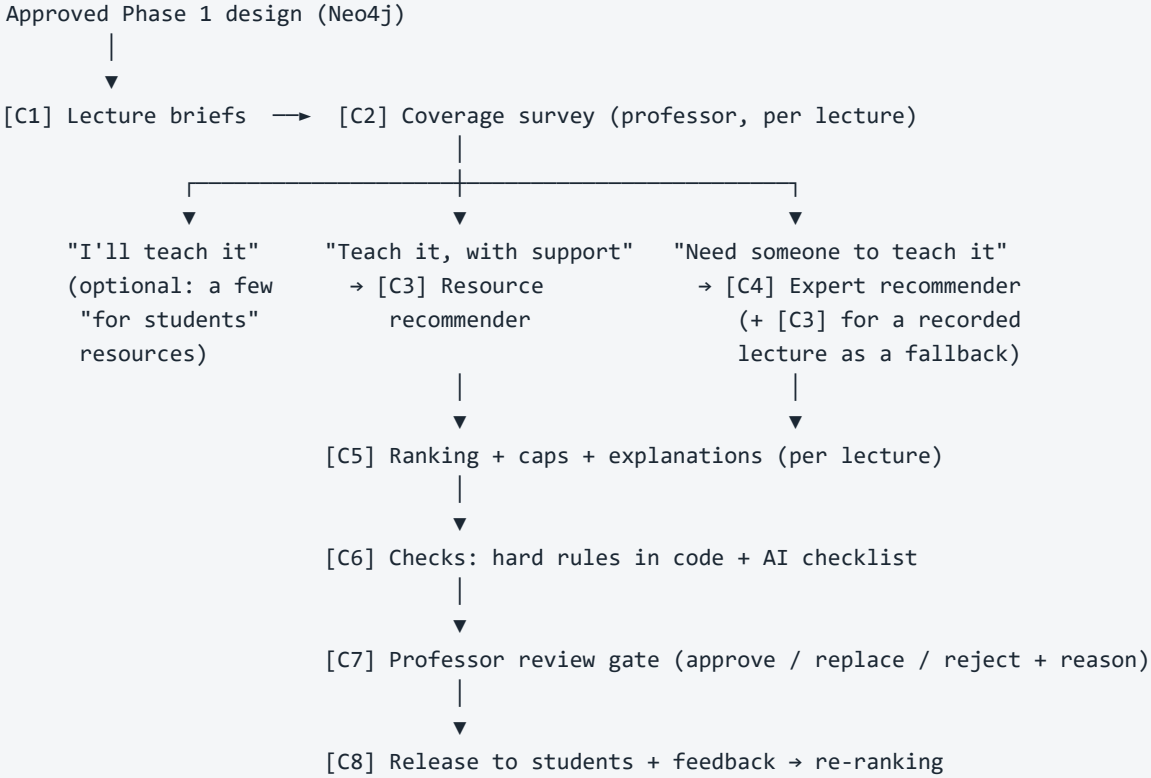
Deliverable	Owner	What it produces
A — UI and simulations	Amit	Professor-facing screens; interactive simulations and visuals
B — Lecture content	Parth	Slides, notes, worked problems, generated content, use of the professor's own material
C — Resources and experts	**Vihan**	**Lecture briefs + coverage survey; ranked external learning resources; ranked expert speakers; guest-lecture kit; "what's new" watch**

Interfaces with the others

- **A (UI):** C provides data and actions (brief, survey answers, recommendation cards, approve/replace/reject); A renders them. C never builds its own UI beyond a CLI/test harness.
- **B (content):** C's approved resources become *sources* B can cite in slides ("retrieve before generate", decision D6). B's generated questions can be paired with C's videos as engagement tasks (D7).
- **Phase 2 architecture mapping:** C owns aid **A3 (books and chapters)** and **A4 (expert videos)**, generalised into one *Learning-resources* aid, and adds a new aid **A9 (expert speakers)**. *(To confirm with the team.)*

Out of scope for C: generating slides or simulations; contacting experts on the professor's behalf; paid-content purchasing; anything about students' personal data.

2. End-to-end workflow



C1. Lecture brief (no help asked for yet)

Built **deterministically from the approved design graph**, with at most one short AI call to phrase a 2-sentence summary:

- lecture number, week, duration; module and topics (with the hours planned);
- the learning outcomes it serves, with Bloom level, and the course outcome each belongs to;
- **what students must already know** (prerequisite topics from earlier weeks and from the prerequisite IIIT-D course, e.g. CSE201);
- **what depends on it** (later lectures, labs, assignments, exams that assess these LOs, with weeks);
- key terms; the lab or tutorial linked to it; the assessment items it feeds;
- a *difficulty / novelty flag*: topics with no match in the IIIT-D knowledge graph or that are very recent (likely to be hardest for the professor).

C2. Coverage survey (the key step before anything else)

For every lecture the professor answers one question, with sensible defaults and bulk actions (per week, per module):

Option	Meaning	What C does
A. I'll teach it	Comfortable	Nothing by default; optional "student resources" (2 items)
B. I'll teach it, with support	Wants preparation and student material	Resource recommender (C3)

Option	Meaning	What C does
C. I need a co-teacher / guest	Wants an expert to deliver all or part	Expert recommender (C4) + resources
D. Use a recorded lecture	Wants a high-quality existing lecture as a flipped class	Resource recommender, full-lecture videos only, plus discussion questions

Plus optional fields: **confidence 1–5**, **note** ("students struggle with X"), **preferred formats** (video / reading / interactive), **max items**. Answers become `HelpRequest` nodes (architecture input I3/I4). The survey is re-openable at any time; a lecture's recommendations are only produced when requested (bounded cost, decision D2).

C3–C8: see §3 (resources), §4 (experts), §6 (checks), §7 (review and feedback).

---

3. Learning-resource recommender

3.1 What can be recommended (resource types)

Type	Examples of sources	Typical use
Full university lectures	MIT OCW, Stanford Online / YouTube, CMU, Berkeley, **NPTEL (IITs/IISc)**, CS50	Flipped class; professor's own preparation
Short explainer videos	3Blue1Brown, StatQuest, Computerphile, freeCodeCamp, official channels of tools	Pre-class primer; in-class clip
Video **segments**	Timestamped part of a long lecture covering exactly one topic	"Watch 12:30–31:00 for X"
Open textbooks and chapters	OpenStax, *Dive into Deep Learning*, open course notes, library books (OpenLibrary/Google Books)	Reading; slide sources for B
Lecture notes / slides of other universities	From the Phase 1 comparable courses (Stanford, MIT, CMU, NUS…)	Professor preparation; reference
Blogs, tutorials, official documentation	Engineering blogs of companies, framework docs	Practical/hands-on topics
Interactive material	Colab/Jupyter notebooks, visualisers (VisuAlgo, PhET), playgrounds	Practice; coordinate with Amit's simulations
Papers and surveys	arXiv, Crossref, OpenAlex (survey and tutorial papers)	Advanced or research-led lectures
Talks, podcasts	Conference talks (NeurIPS, ICSE, PyCon, Strange Loop…), podcasts	Real-world perspective; also feeds the expert list
Practice and datasets	Problem sets of open courses, Kaggle datasets, GitHub exercises	Assignments, labs
Industry case studies	Post-mortems, architecture write-ups	Motivation, "why it matters"

3.2 Where candidates come from (sources, tiered)

Sources are configured in `config/resources.yaml` with a **tier** used in ranking and a whitelist/blocklist (the Phase 1 blocklist for held-out courses still applies):

- **Tier 1 — universities and national programmes:** MIT OCW, Stanford, CMU, Berkeley, Harvard (CS50), Oxford/Cambridge, NUS, ETH, **NPTEL / IITs**, IIIT-D's own courses (the knowledge graph's resources and prerequisite courses).
- **Tier 2 — established educators:** curated channel list (freeCodeCamp, 3Blue1Brown, StatQuest, Computerphile, Two Minute Papers, etc.), recognised MOOC instructors.
- **Tier 3 — practitioners and official docs:** company engineering blogs, official documentation, maintainers' tutorials, conference channels.
- **Tier 4 — general web:** allowed only when tiers 1–3 cannot cover a topic; shown with a "general source" label.

3.3 How candidates are found (discovery)

1. **Query planning (1 AI call per lecture):** from the brief, write 3–6 search queries per lecture (topic phrases, LO verbs, synonyms, "lecture", "tutorial", course-level hints). Same pattern as Phase 1's X1/X3 prompts.
2. **Adapters (all behind the integration layer, with caching and quotas):**
    - **YouTube Data API v3:** `search.list` (100 units; ~100 searches/day on the free 10,000-unit quota) to find videos; `videos.list` (1 unit per call, 50 videos per call) for duration, views, likes, comments count, publication date, captions flag, licence (YouTube / Creative Commons), region restrictions, embeddability; `playlistItems` for lecture series. Searches are cached for weeks; university playlists are indexed once and reused.
    - **Grounded web search** (Gemini, restricted to whitelisted domains, as in Phase 1) for course pages, OCW lecture pages, NPTEL, blogs and docs.
    - **Phase 1 research results:** the comparable courses found at Stanford/MIT/NUS already list their lectures, readings and textbooks; those are high-quality, topic-aligned candidates at no search cost.
    - **IIIT-D knowledge graph:** resources used by related IIIT-D courses (institutional continuity).
    - **Books:** OpenLibrary / Crossref (already used and verified in Phase 1), Google Books for chapter titles.
    - **Papers:** OpenAlex (free key), arXiv, Crossref — preferring surveys and tutorials.
    - **Code / interactive:** GitHub search (stars, recent activity, licence), notebook collections.
3. **Enrichment:** fetch metadata; for videos fetch the **transcript** (run locally — the common Python transcript library is blocked from cloud servers but generally works from a home or campus network; fallback: title + description + chapter markers); for pages fetch the text.

3.4 How candidates are ranked

A transparent weighted score per candidate, shown to the professor as a breakdown (weights in config, tuned in evaluation):

Criterion	Weight	How it is measured
Topic coverage	0.35	Fraction of the lecture's topics and LOs the resource actually covers: transcript/page chunks embedded and matched to each topic (the local embedding model already used for the knowledge graph). Produces a per-topic coverage map and, for videos, **timestamps** per topic.
Explanation quality	0.20	AI rubric on a transcript sample: clarity, correctness, depth, use of examples, structure (the "relevance, accuracy, clarity, engagement, depth" style of rubric). Fixed rubric, scored twice to reduce run-to-run variance; recorded as *judgement*.
Source authority	0.15	Source tier; channel/author on the curated list; author credentials (links to C4's expert evidence).

Criterion	Weight	How it is measured
Level fit	0.10	Course level and the brief's prerequisite knowledge vs the resource's assumed background (AI check + simple cues such as "introduction", "advanced").
Audience reception	0.10	Like/view ratio, comment volume, views normalised by age (log scale), sampled comment sentiment; for MOOCs, public ratings where available. Capped so popularity cannot dominate.
Practicality	0.10	Duration fits the use (clip ≤ 20 min, full lecture ≤ 90 min); captions available; recency for fast-moving topics; licence permits the intended use; available in India; free.

Then:

- **Diversity re-ranking (MMR):** avoid three items from the same channel or of the same type; aim for a mix (e.g. one full lecture, one short explainer, one reading, one practice).
- **Caps (defaults, professor-adjustable):** per lecture **at most 3 videos, 2 readings, 1 interactive/practice item**; a "show more" action reveals the next few on demand.
- **Explanation per item:** one line saying *why* it is recommended, which topics it covers (with timestamps), the level, the time needed, and **how to use it** (before class / in class / after class / professor preparation).
- **Reviews:** a 2–3 line review per item combining the rubric findings with public reception ("Clear derivations, good examples; 98% positive reactions; skips topic X; assumes linear algebra").

3.5 Output format (per lecture)

Illustrative shape only (not real search results):

```
Lecture W02.L1 · Spec-driven development tools · 2 h · LOs: CO2.LO1, CO2.LO2
Professor's choice: B (teach with support) · note: "students confuse specs with tests"

VIDEOS
1. ★ 0.86  <title> — <university/channel> · 54 min · CC BY-NC-SA · captions ✓
   Covers: executable specifications (03:10–21:40), spec vs test (21:40–35:05)  ·  Missing: tool setup
   Why: university lecture, covers both LOs, clear worked example; level fits 3rd year
   Use: professor preparation + flipped-class option
2. ★ 0.79  <short explainer> … 14 min · Use: in-class clip for "spec vs test"
3. ★ 0.71  <conference talk by a practitioner> … Use: after class, real-world view
READINGS
1. ★ 0.82  <book, chapter 4> (verified record) · Use: reading for LO1
2. ★ 0.74  <official documentation guide> · Use: lab preparation
PRACTICE
1. ★ 0.77  <open notebook / exercise set> · Use: tutorial
(3 more available)
```

---

4. Expert and practitioner recommender

4.1 Who can be recommended

Academics (professors, researchers), **industry practitioners** (engineers, architects, product people), educators and well-known instructors, open-source maintainers, book authors and frequent conference speakers — **anywhere in the world**, with IIIT-D faculty, alumni and Indian experts surfaced alongside international ones.

4.2 Where candidates come from

Source	What it gives	Notes
OpenAlex	Authors by topic, works, citations, recent activity, institution, ORCID	Free key; ~$1/day free budget is plenty
DBLP	Computer-science publication records	Free; good for CS name disambiguation
Semantic Scholar	Author profiles, h-index, influential citations	Free; strict rate limits — use a key and back off
ORCID	Verified identity and affiliations	Public API
Talks and lectures found in C3	Speakers of top-ranked lectures and conference talks	Direct evidence of *teaching ability*
GitHub	Maintainers of key repositories, activity	For practitioner expertise
Books	Authors of recommended books (OpenLibrary)	
Grounded web search	Institutional profile pages, conference speaker pages, company engineering blogs	Only public professional pages
IIIT-D knowledge graph	IIIT-D faculty who teach related courses	Internal experts first-class

Not used: LinkedIn or any source whose terms forbid automated access; personal social media; private contact details.

4.3 How experts are ranked

Criterion	Weight	Evidence
Topical fit	0.35	Their works / talks / repos matched to the lecture's topics and LOs (embeddings); recent work weighted higher
Teaching and communication evidence	0.20	Recorded lectures or talks (and how well C3 ranked them), courses taught, tutorials, books
Practical / industry relevance	0.15	Hands-on work for applied topics (repos, products, engineering posts); weight shifts by lecture type (theory vs practice)
Standing	0.10	Field-normalised citations, invited talks; capped so the list is not just the most famous names
Recency and activity	0.10	Active in the last 2–3 years
Feasibility	0.10	Time-zone overlap with IST, prior remote talks, language, IIIT-D connection (alumni/collaborators)

Output per lecture: top **5** (adjustable), each with: name, role and affiliation, **why this person for this lecture** (two lines tied to specific evidence), 2–4 evidence links (paper, talk, repo, profile), suggested format (live remote 60 min / recorded / panel / Q&A after a recorded lecture), time-zone note, and a public professional contact page. A **diversity check** keeps a mix of academia and industry and of regions. The system never infers or uses gender, ethnicity or other personal attributes.

4.4 Guest-lecture kit (once the professor picks someone)

- **Brief for the speaker:** course and audience (programme, year, prerequisites), the exact topics and LOs, what students saw before and what comes after, duration, related lab/assessment, any professor note.
- **Invitation email draft** in the professor's name, for the professor to edit and send (the system never contacts anyone itself).

- **Fallbacks:** the speaker's best recorded talk on the topic (from C3) as a backup, and two alternative speakers.
- **After the lecture:** record "taught by", the recording link if consent is given, and feedback (feeds the coverage tracker of the architecture, input I8).

4.5 Ethics and privacy

Public professional information only; minimal storage (name, affiliation, public profile links, evidence links); no automated outreach; opt-out honoured (a removed person is never suggested again); the professor approves every list; the reasoning shown cites evidence so it can be checked.

---

5. More deliverables that would help the professor (research)

Ranked by value for the "professor can only teach part of the course" problem. **Bold = proposed for C; others to coordinate.**

#	Deliverable	Why it helps	How	Owner
1	**Professor up-skilling path**	The professor may not know a topic well	Per weak lecture: 2–3 resources *for the professor*, ordered, with time estimate ("3 h to prepare")	C
2	**Video segment playlists**	Long lectures waste time	Topic-to-timestamp map from transcripts; "clip playlist" per lecture	C
3	**"What's new" watch**	Fast-moving fields change every few months	For flagged topics: recent papers, releases, notable posts in the last 6–12 months; monthly refresh	C
4	**Prerequisite refresher pack**	Students arrive with gaps	From the brief's prerequisites: short refreshers, IIIT-D prerequisite course material	C
5	**Flipped-class pack**	A recorded lecture needs structure	Chosen video + 3–5 discussion questions + one in-class activity	C + B (questions)
6	**Industry case-study bank**	"Why does this matter?"	Curated engineering write-ups and post-mortems per module	C
7	**Resource health monitor**	Links rot, videos get removed	Weekly re-check of approved links; replacement suggestions	C
8	**Licence and reuse note**	What can be shown, downloaded or adapted	Licence recorded per item; "link only" vs "may adapt"	C
9	**Practice and dataset finder**	Labs and assignments need material	Open problem sets, datasets, notebooks matched to LOs	C (coordinate with B)
10	Engagement tasks for videos	Watching alone is passive (D7)	Predict–watch–explain questions per video	B
11	Accessibility check	Captions, language	Captions flag, Hindi/English availability	C (data) + A (display)
12	Panel / co-teaching planner	Some topics suit a panel	Pair 2–3 experts with complementary views	C (later)

---

6. Checks before the professor sees anything (requirements R4, R5)

Same engine as Phase 1: hard rules in code produce findings with severity and a fix hint; an AI checklist adds judgement findings; failing items are replaced by the next candidate (repair = re-select, at most 2 rounds).

Hard rules (code)

- Link resolves; video exists, is public, embeddable and not region-blocked in India; duration and date known.
- Source allowed (whitelist / not blocklisted); held-out test courses never appear.
- Claimed topic coverage is supported: each claimed topic has transcript/page evidence above the similarity threshold; timestamps lie inside the video.
- Duration fits the declared use; caps respected; no duplicates (same video via two URLs, same book twice).
- Licence recorded; captions flag recorded (missing captions → warning).
- Books and papers verified against a library record (Phase 1's verifier).
- Experts: identity confirmed in at least one authoritative source (OpenAlex / ORCID / DBLP / institutional page); every evidence link resolves; no personal contact data stored.

AI checklist (judgement)

- Does the resource *explain* the LO, not just mention it?
- Is it at the right level for the cohort?
- Any visible inaccuracies or outdated content?
- For experts: is the reason grounded in the cited evidence?

As in Phase 1, judgement findings are shown as advice with a suggested fix; whether they block is the supervisor's open question (§16.2 of the architecture).

---

7. Review, release and feedback

- **Review gate (C7):** per lecture, the professor sees the ranked cards and can approve, **replace with the next candidate**, reject with a reason, ask for "more like this", or pin their own link. Approved items are frozen and linked in the graph: `(Lecture)-[:USES]->(Material)-[:COVERS_LO]->(LO)`; experts: `(Lecture)-[:CANDIDATE_SPEAKER {rank, score}]->(Expert)`.
- **Release (C8):** only approved items go to students, through the architecture's release service.
- **Feedback loop:** professor decisions and student ratings (1–5, "helpful for topic X") adjust per-source and per-criterion weights; repeatedly rejected sources drop in tier; recurring complaints become new hard rules (the Phase 1 practice).

---

8. Architecture and data

8.1 Components (fit the Phase 2 layers)

Layer	Component (C)
L2 Application	Coverage-survey service; resource/expert planner (orders work by teaching week, within budget)
L3 Aid services	`LearningResourcesAid` (A3+A4), `ExpertSpeakerAid` (A9) — both implement `produce(WorkItem, context) -> Material` and declare their checks and AI-call cost

Layer	Component (C)
L4 Validation	Resource rules + expert rules + AI checklists (Phase 1 engine)
L5 Integration	Adapters: YouTube Data API, transcripts, grounded search, OpenAlex, DBLP, Semantic Scholar, ORCID, GitHub, OpenLibrary/Crossref — each with cache, quota and fallback
L6 Data	Neo4j nodes below; cached search results and transcripts in `data/cache/`

8.2 New graph nodes and links

- `LectureBrief {lecture_id, summary, prerequisites, dependents, novelty_flags}`
- `CoverageDecision {lecture_id, option A–D, confidence, note, formats, decided_by, at}` → becomes `HelpRequest`
- `Recommendation {kind: video|reading|interactive|practice|paper|talk, url, title, source, tier, licence, duration, published, scores{…}, total, rank, rationale, review, use_as, status}` with `(:Recommendation)-[:FOR_LECTURE]->(Lecture)`, `-[:COVERS {start, end, sim}]->(Topic)`, `-[:COVERS_LO]->(LO)`
- `Expert {name, affiliation, role, openalex_id, orcid, dblp, github, profile_url}` with `(:Expert)-[:CANDIDATE_FOR {rank, score, rationale, format}]->(Lecture)` and `(:Expert)-[:EVIDENCED_BY]->(Evidence {kind, url, title})`
- `Feedback` attaches to any of these (architecture archetype).

8.3 Configuration

`config/resources.yaml` (sources, tiers, whitelist, caps, weights, duration limits, recency rules), `config/experts.yaml` (sources, weights, diversity rules, top-N). New keys in `.env`: `YOUTUBE_API_KEY`, `OPENALEX_API_KEY`, `S2_API_KEY` (optional), `GITHUB_TOKEN` (optional).

8.4 Budget (free tier)

Per lecture that asks for help (typical):

Item	Cost
Query planning	1 AI call
YouTube searches	2–3 × 100 units (of 10,000/day) + 1–2 units for details
Transcripts, embeddings, coverage	free (local)
Quality rubric for the top ~6 candidates	1–2 AI calls (batched)
Rationales and reviews	1 AI call
Experts (if option C)	OpenAlex/DBLP calls (cheap) + 1–2 AI calls

So **~4–6 AI calls and ~300 YouTube units per lecture**: roughly 30 lectures/day on YouTube quota, and a whole course in 1–2 days with caching, if a professor asks for help on many lectures. Indexing the main university playlists once removes most repeat searches.

---

9. How we will show it works (evaluation)

- **Gold set:** for ~10 lectures from CSE201 and CS-601, two people (ideally faculty) label candidate resources and experts as *excellent / usable / not suitable*.

- **Metrics:** precision@3 and nDCG@5 for resources and experts; LO coverage of the top-3; % of links valid (target 100% after checks); % of items rejected by the professor; time a professor spends per lecture.
- **Baselines:** (a) YouTube's own top-3 for the lecture title; (b) asking an LLM directly for links (expect invented or dead links); (c) our ranking without coverage (to show what transcript coverage adds).
- **Expert list:** precision@5 by professor judgement; diversity (academia vs industry, regions).
- **Pilot:** one professor, one course, a few weeks of real use; collect decisions and student ratings.

---

10. Execution plan

Milestone	Duration	Output
M0 · Contracts	2–3 days	Agree with Amit/Parth: data model, aid interface, UI actions, which aids C owns; get API keys
M1 · Briefs + survey	1 week	Lecture briefs from the design graph; coverage-survey service and graph nodes; CLI harness; tested on CS-601 and CSE201
M2 · Discovery	1.5 weeks	Adapters (YouTube, transcripts, grounded search, OpenAlex/arXiv, books, GitHub) with cache and quotas; query planner; reuse of Phase 1 comparable-course materials
M3 · Ranking + checks	1.5 weeks	Coverage mapping with timestamps, rubric, scoring, diversity, caps, rationales and reviews; hard rules + AI checklist; replacement on failure
M4 · Experts	1.5 weeks	Candidate sourcing, disambiguation, ranking, evidence, guest-lecture kit
M5 · Gate + feedback	1 week	Review actions wired to A's UI; release; feedback re-weighting; resource health monitor
M6 · Evaluation	1 week	Gold set, metrics vs baselines, pilot report
Later	—	Up-skilling paths, "what's new" watch, case-study bank, panel planner

---

11. Risks and mitigations

Risk	Mitigation
YouTube quota (~100 searches/day)	Cache aggressively; index university playlists once; use Phase 1 course materials and grounded search first
Transcripts unavailable or blocked	Run locally; fall back to description, chapters and title; mark coverage "estimated"
Popularity bias (famous = good?)	Coverage first; reception capped; tier and rubric separate from views
AI rubric varies run to run	Fixed rubric, two samples averaged, shown as judgement; professor decides
Licences and terms of use	Link and embed only, never download or redistribute; licence shown per item; follow YouTube API display rules
Expert privacy	Public professional data only; no outreach by the system; opt-out list
Western-centric lists	NPTEL/IIT and Indian experts in tier 1; diversity rule across regions
Link rot	Weekly health monitor and replacements
Overlap with B/A	Contracts agreed in M0; C provides data, A renders, B cites

---

12. Decisions needed (your signals)

1. **Ownership:** C owns aids A3 (books) + A4 (videos) as one resource aid, plus a new expert aid (A9)?
2. **Caps:** 3 videos + 2 readings + 1 interactive per lecture, and top-5 experts, as defaults?
3. **Weights:** coverage-first weights as in §3.4 and §4.3, or a different emphasis (e.g. more on authority)?
4. **Free only?** Exclude paid MOOCs and books behind paywalls by default?
5. **Expert scope:** global with an IIIT-D/India mix, academia + industry, as proposed? Any exclusions?
6. **Survey options:** are the four options (teach / teach with support / guest / recorded lecture) right for professors?
7. **Extras:** which of §5's additions to build first (proposal: up-skilling path, segment playlists, "what's new" watch)?
8. **Pilot:** which course and professor?

Sources consulted for the external-service facts

- YouTube Data API quota and costs: https://developers.google.com/youtube/v3/determine_quota_cost, https://developers.google.com/youtube/v3/getting-started
- Transcript access from cloud vs local networks: https://github.com/jdepoix/youtube-transcript-api, https://github.com/jdepoix/youtube-transcript-api/issues/593
- OpenAlex keys and limits: https://developers.openalex.org/api-reference/authentication, https://docs.openalex.org/how-to-use-the-api/rate-limits-and-authentication
- Semantic Scholar API: https://semanticscholar.org/product/api
- MIT OCW licence (CC BY-NC-SA): https://support.learn.mit.edu/hc/en-us/articles/42909882604315-What-are-the-requirements-of-use-for-MIT-OpenCourseWare
- Educational video quality evaluation (rubrics, risks of LLM scoring): https://arxiv.org/html/2407.12005v2, https://www.jsr.org/hs/index.php/path/article/view/6107
- Academic identity resolution via OpenAlex/ORCID: https://help.openalex.org/how-to/finding-openalex-ids/